

Spotify Discord Battle Bot - Complete Development Roadmap

PROJECT IDEA:

A Discord bot integrated with Spotify where 2-5 users battle by submitting songs.

Users anonymously rate songs, scores are calculated, and leaderboards/profiles are created.

=====

PROJECT STRUCTURE:

spotify_battle_bot/

main.py

- Starts Discord bot
- Loads extensions
- Handles startup

config.py

- Environment variables
- API keys

api.env

- Discord token
- Spotify client ID
- Spotify secret

cogs/

battle.py

/battle_create

join buttons

voting system

spotify.py

Spotify commands

song search

profile.py

user stats

services/

spotify_api.py

OAuth

Spotify API requests

Track metadata

database/

models.py

User

Battle

Song tables

utils/

helpers.py

=====

PHASE 0 - DISCORD FOUNDATION

Completed:

- discord.py setup
- Bot class
- Slash commands
- Embeds
- Buttons

- Global commands

Learn next:

- Cogs
- Modular architecture

=====

PHASE 1 - SPOTIFY INTEGRATION

Goal:

Connect Discord with Spotify.

Tasks:

1. Create Spotify developer app

2. Get:

- Client ID
- Client Secret

3. Implement OAuth

Commands:

/spotify_login

/song

Flow:

Discord

|

Spotify API

|

Track JSON

|

Discord Embed

Data collected:

- Track name
- Artist
- Album image
- Popularity
- Spotify link

=====

PHASE 2 - MUSIC BATTLE SYSTEM

Main Feature.

Commands:

/battle_create

Creates:

Battle Lobby

Players:

1-5

Buttons:

JOIN

LEAVE

START

Data:

```
battle = {  
  players:[],  
  songs:[],  
  votes:{},  
  status:"waiting"  
}
```

=====

SONG SUBMISSION

Command:

```
/submit_song spotify_link
```

Flow:

Spotify URL

|

Extract track ID

|

Spotify API

|

Store song

=====

VOTING SYSTEM

Songs displayed anonymously.

Example:

Song #1

Rate:

1 Star

2 Star

3 Star

4 Star

5 Star

Store:

```
votes = {  
  song_id:{  
    user:rating  
  }  
}
```

=====

RESULT SYSTEM

Calculate:

Average = total score / voters

Leaderboard:

1st place

Song

User

Score

2nd place

3rd place

=====

PHASE 3 - DATABASE

Start:

SQLite

Upgrade:

PostgreSQL

Tables:

USERS

discord_id

spotify_id

wins

losses

rating

BATTLES

battle_id

server_id

winner

date

SONGS

spotify_id

score

times_used

=====

PHASE 4 - UNIQUE FEATURES

1. Music Ranking System

Bronze Listener

Silver DJ

Gold DJ

Radiant DJ

2. AI Judge

Uses LLM:

Inputs:

Songs

Theme

Ratings

Output:

AI opinion

3. Server Wrapped

/server_wrapped

Shows:

Top songs

Top artists

Best music taste

=====

PHASE 5 - DEPLOYMENT

Tools:

GitHub

Docker

Cloud hosting

Options:

Railway

Render

VPS

Final result:

Bot runs 24/7

=====

PROGRESS TRACKER

Discord Basics:

COMPLETE

Spotify API:

TODO

Battle Logic:

TODO

Database:

TODO

AI Features:

TODO

Deployment:

TODO

Recommended Build Order:

1. Restructure project
2. Learn Discord cogs
3. Spotify API connection
4. Song search command
5. Battle creation
6. Join system
7. Voting
8. Leaderboard
9. Database

10. AI features

11. Deployment